

**Laporan Praktikum Modul III: Event, State, Imperatif-Deklaratif, dan
Implisit Intent Pada Jetpack Compose**
Tugas Mata Pengembangan Aplikasi Mobile

Dosen Pengampu :
Muhammad Aminul Akbar, S.Kom., M.T



Disusun Oleh:

Nazwa Hafidzah

245150407111023

**SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026**

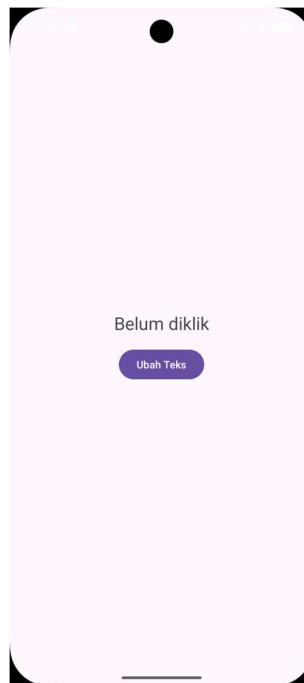
Percobaan 1: Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

Pada percobaan ini dibuat dua project dengan hasil akhir yang sama, yaitu teks “Belum diklik” yang berubah menjadi “Tombol sudah diklik” setelah tombol “Ubah Teks” ditekan. Project A (ImperativeXmlApp) dibangun dengan XML/View yang bersifat imperatif, sedangkan Project B (DeclarativeComposeApp) dibangun dengan Jetpack Compose yang bersifat deklaratif. Hal yang dianalisis bukan hasil akhirnya, melainkan cara masing-masing pendekatan memperbarui tampilan.

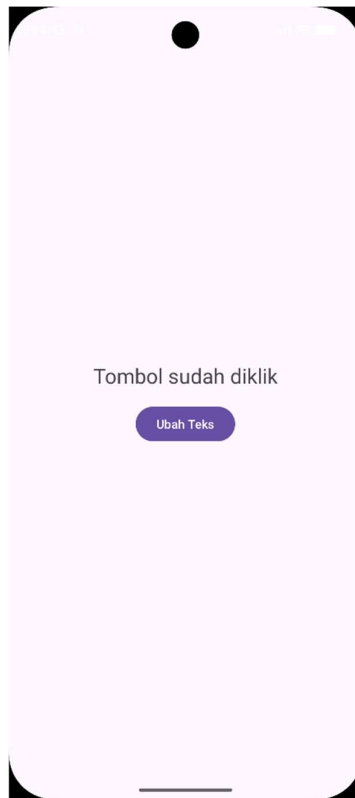
1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Jawaban:

Berdasarkan hasil percobaan, kedua aplikasi menampilkan teks “Belum diklik” beserta tombol “Ubah Teks” pada kondisi awal. Setelah tombol ditekan, teks pada kedua aplikasi berubah menjadi “Tombol sudah diklik”. Tampilan akhir yang dihasilkan sama persis, namun mekanisme yang bekerja di balik perubahan tersebut berbeda, dan perbedaan itu dibahas pada soal-soal berikutnya.



Gambar 1. Sebelum tombol diklik



Gambar 2. Sesudah tombol diklik

2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban:

Pada project Compose, nilai yang diubah adalah state message, bukan komponen Text itu sendiri. State tersebut dibuat menggunakan remember dan mutableStateOf, kemudian nilainya diganti di dalam lambda onClick.

```
var message by remember {  
    mutableStateOf("Belum diklik")  
}  
  
Text(text = message, fontSize = 24.sp)  
  
Button(onClick = {  
    message = "Tombol sudah diklik"  
}) {  
    Text("Ubah Teks")  
}
```

Baris yang dijalankan ketika tombol diklik adalah `message = "Tombol sudah diklik"`. Baris ini hanya mengubah data, sedangkan composable `Text(text = message)` tidak disentuh secara langsung. Teks pada layar ikut berubah karena composable tersebut membaca state `message`.

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.

Jawaban:

Pada project Compose, nilai yang diubah adalah state `message`, bukan komponen `Text` itu sendiri. State tersebut dibuat menggunakan `remember` dan `mutableStateOf`, kemudian nilainya diganti di dalam lambda `onClick`.

```

var message by remember {
    mutableStateOf("Belum diklik")
}

Text(text = message, fontSize = 24.sp)

Button(onClick = {
    message = "Tombol sudah diklik"
}) {
    Text("Ubah Teks")
}

```

Baris yang dijalankan ketika tombol diklik adalah `message = "Tombol sudah diklik"`. Baris ini hanya mengubah data, sedangkan composable `Text(text = message)` tidak disentuh secara langsung. Teks pada layar ikut berubah karena composable tersebut membaca state `message`.

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Jawaban:

Perintah seperti `tvMessage.text` atau `setText()` tidak ditemukan pada Compose karena `Text()` merupakan fungsi composable yang mendeskripsikan tampilan, bukan objek View yang disimpan dan dapat direferensikan seperti `tvMessage`. Oleh karena itu, tidak ada objek yang dapat diperintah untuk mengganti teksnya.

Selain itu, Compose menggunakan pendekatan deklaratif dengan prinsip $UI = f(state)$, yaitu tampilan merupakan hasil dari state saat ini. Karena itu, yang perlu diubah cukup sumber datanya, yaitu state. Ketika state berubah, Compose sendiri yang menjalankan recomposition dan memanggil ulang composable yang bergantung pada state tersebut, sehingga pengembang tidak perlu mencari komponen dan memerintahkannya berubah. Pendekatan ini juga membuat tampilan lebih terjaga kesesuaiannya dengan data, sebab hanya terdapat satu sumber kebenaran, yaitu state.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Jawaban:

Alur perubahan pada XML/View berlangsung sebagai berikut. Pertama, pengguna menekan tombol `btnChange` sehingga event klik terjadi dan `setOnClickListener` dijalankan. Kedua, di dalam listener tersebut program mengakses objek View `tvMessage` yang telah diperoleh melalui `findViewById`, lalu memerintahkannya dengan `tvMessage.text = "Tombol sudah diklik"`. Ketiga, objek `TextView` memperbarui properti `text`-nya dan menggambar ulang dirinya, sehingga teks pada layar berubah menjadi "Tombol sudah diklik".

Jadi, polanya adalah event menyebabkan program mencari dan memerintahkan View, kemudian View mengubah tampilannya.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Jawaban:

Alur perubahan pada Compose berlangsung sebagai berikut. Pertama, pengguna menekan tombol sehingga lambda `onClick` dijalankan (event). Kedua, di dalam `onClick`, nilai `state message` diubah menjadi "Tombol sudah diklik" (state). Ketiga, Compose mendeteksi bahwa `state message` yang dibuat dengan `mutableStateOf` telah berubah, lalu menjalankan ulang bagian UI yang membaca state tersebut (recomposition). Keempat, `Text(text = message)` dipanggil kembali dengan nilai yang baru sehingga teks pada layar berubah (perubahan tampilan). Jadi, pada Compose event tidak memerintahkan komponen untuk berubah, melainkan mengubah state yang menjadi sumber data bagi tampilan.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jawaban:

Aplikasi tetap dapat dikompilasi dan dijalankan, tetapi teks di layar tidak berubah dan tetap "Belum diklik" walaupun tombol diklik. Kode yang dimaksud:

```

var message = "Belum diklik"

Text(text = message)

Button(onClick = { message = "Tombol sudah diklik" }) {

    Text("Ubah Teks")

}

```

Penyebabnya:

- Variabel biasa memang bisa berubah nilainya, tetapi Compose tidak mengamatinya. Compose hanya mengetahui perubahan pada objek state seperti mutableStateOf.
- Karena Compose tidak diberi tahu ada perubahan, tidak terjadi recomposition. Text tidak dipanggil ulang, sehingga tampilan tidak diperbarui.
- Tanpa remember, variabel juga dibuat ulang dengan nilai awal “Belum diklik” setiap kali fungsi composable dijalankan lagi. Jadi seandainya terjadi recomposition karena sebab lain, nilainya kembali ke awal.

Kesimpulannya, agar UI berubah, data harus disimpan sebagai state (mutableStateOf) dan dipertahankan dengan remember.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Jawaban:

Berdasarkan percobaan yang telah dilakukan, dapat disimpulkan bahwa pendekatan imperatif mengharuskan pengembang mengatur tampilan langkah demi langkah, yaitu dengan mencari komponen UI lalu memerintahkannya berubah. Pengembang bertanggung jawab memastikan tampilan selalu sesuai dengan data. Sebaliknya, pendekatan deklaratif menuntut pengembang hanya mendeskripsikan tampilan yang sesuai untuk suatu state. Ketika state berubah, framework yang memperbarui bagian tampilan yang terkait. Dengan kata lain, pendekatan imperatif berfokus pada bagaimana cara mengubah tampilan, sedangkan pendekatan deklaratif berfokus pada apa yang harus ditampilkan untuk state saat ini. Karena itu, kode deklaratif cenderung lebih ringkas dan risiko ketidaksesuaian antara tampilan dan data menjadi lebih kecil.

Percobaan 2: Event Handling dan State

Percobaan ini membandingkan perilaku composable `EventStateTest` ketika count berupa variabel biasa dengan ketika count berupa state. Pada bagian akhir ditambahkan state text dan `OutlinedTextField` untuk mengamati penanganan input. Perlu dipastikan bahwa import `android.util.Log` sudah ditambahkan agar pemanggilan `Log.d` tidak menimbulkan error.

Bagian A: count sebagai variabel biasa (`var count = 0`)

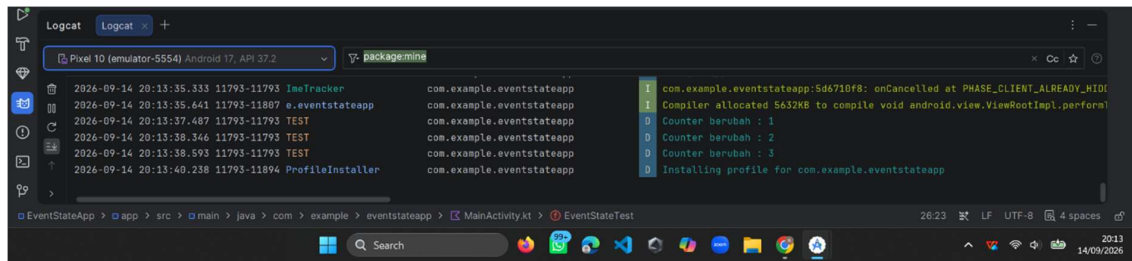
1. Jalankan program, filter LogCat dengan “TEST”, klik tombol “Tambah” beberapa kali, lalu ambil satu screenshot tampilan aplikasi.

Jawaban:

Aplikasi dijalankan, LogCat difilter dengan kata TEST, dan tombol “Tambah” diklik beberapa kali. Hasilnya, tulisan pada layar tetap “Counter : 0” meskipun tombol sudah diklik berulang kali.



Gambar 1. Tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali



Gambar 2. Tampilan Logcat dengan filter TEST

2. Apa yang muncul di LogCat setiap kali tombol “Tambah” diklik?

Jawaban:

Setiap tombol diklik, LogCat menampilkan satu baris log dengan tag TEST dan angka yang terus bertambah, yaitu “Counter berubah : 1”, “Counter berubah : 2”, “Counter berubah : 3”, dan seterusnya. Hal ini menunjukkan bahwa nilai count di dalam lambda onClick memang bertambah pada setiap klik.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol “Tambah” diklik?

Jawaban:

Tidak terjadi perubahan pada tampilan aplikasi. Teks tetap menunjukkan “Counter : 0” berapa pun jumlah klik yang dilakukan

4. Apa yang menyebabkan hal tersebut terjadi?

Jawaban:

Hal tersebut terjadi karena count dideklarasikan sebagai variabel biasa (`var count = 0`), bukan sebagai state. Nilainya memang bertambah, sebagaimana terlihat pada LogCat, tetapi Compose tidak mengamati variabel biasa. Karena tidak ada state yang berubah, recomposition tidak terjadi sehingga `Text("Counter : $count")` tidak dipanggil ulang dan layar tetap menampilkan nilai lama.

Bagian B: count sebagai state

9. Apa yang muncul di LogCat setiap kali tombol “Tambah” diklik?

Jawaban:

LogCat tetap menampilkan “Counter berubah : 1”, “Counter berubah : 2”, “Counter berubah : 3”, dan seterusnya. Isi log ini sama dengan percobaan sebelumnya karena count++ dan Log.d tetap dijalankan pada setiap klik.

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol “Tambah” diklik?

Jawaban:

Tampilan aplikasi berubah. Setiap kali tombol diklik, angka pada teks “Counter : ...” bertambah satu, yaitu dari 0 menjadi 1, kemudian 2, 3, dan seterusnya.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Jawaban:

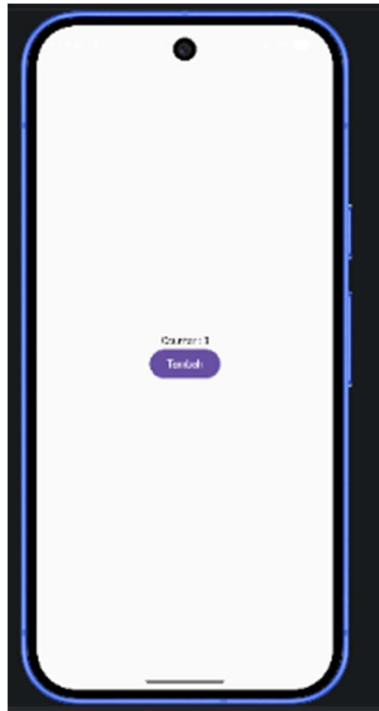
Perbedaan tersebut disebabkan oleh jenis variabel count. Pada percobaan sebelumnya count merupakan variabel biasa yang tidak diamati oleh Compose. Pada percobaan ini count dibuat dengan mutableStateOf sehingga perubahannya dapat diamati. Ketika count++ dijalankan, Compose mengetahui bahwa state telah berubah, lalu menjalankan recomposition pada composables yang membaca count, sehingga Text menampilkan nilai yang baru.

Fungsi remember menyimpan nilai state selama composable masih berada dalam composition, sehingga nilai count tidak kembali ke 0 setiap kali recomposition terjadi. Adapun penggunaan by (property delegation) membuat count dapat dipakai langsung tanpa harus menulis .value.

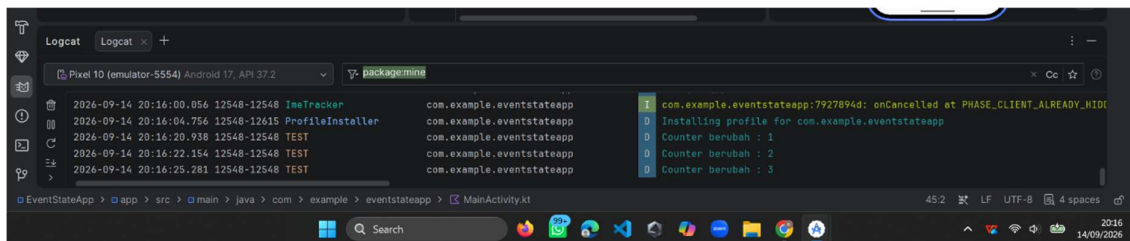
12. Apa yang dimaksud recomposition pada Compose?

Jawaban:

Recomposition adalah proses ketika Compose menjalankan ulang fungsi composables yang membaca suatu state setelah nilai state tersebut berubah, sehingga tampilan diperbarui sesuai dengan nilai terbaru. Compose berusaha hanya menjalankan ulang bagian UI yang terpengaruh oleh perubahan state, bukan seluruh tampilan aplikasi.



Gambar 1. Tampilan aplikasi setelah tombol “Tambah” diklik beberapa kali (versi state)



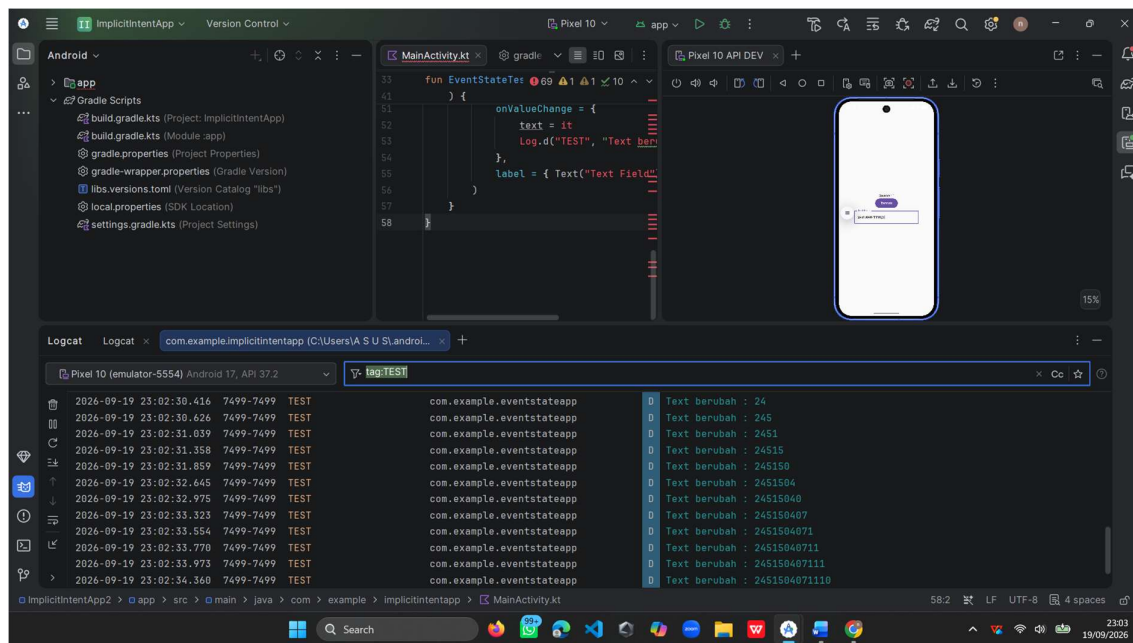
Gambar: Tampilan LogCat dengan filter TEST (versi state)

Bagian C: penambahan state text dan OutlinedTextField

14. Jalankan kembali, filter LogCat dengan “TEST”, lalu ambil screenshot tampilan aplikasi.

Jawaban:

Aplikasi dijalankan kembali, tombol “Tambah” diklik beberapa kali, dan beberapa huruf diketik pada Text Field. Layar menampilkan angka counter beserta teks yang diketik pada field.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

Jawaban:

Setiap satu huruf diketik, LogCat menampilkan satu baris log “Text berubah : ...” yang berisi seluruh isi teks pada saat itu, bukan hanya huruf yang terakhir diketik. Sebagai contoh, apabila diketik “abc”, log yang muncul secara berurutan adalah “Text berubah : a”, “Text berubah : ab”, dan “Text berubah : abc”. Penghapusan huruf juga memunculkan log, misalnya ketika “abc” menjadi “ab”.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Jawaban:

Huruf yang diketik langsung tampil di dalam Text Field. Nilai counter tidak berubah karena kegiatan mengetik tidak memengaruhi state count.

17. Apa yang menyebabkan hal tersebut terjadi?

Jawaban:

Setiap kali isi field berubah, onValueChange dipanggil dengan membawa nilai baru (it). Nilai tersebut disimpan ke dalam state melalui text = it, kemudian log dicetak. Karena value = text membaca state itu, perubahan state memicu recomposition sehingga OutlinedTextField menampilkan nilai yang baru. Hal ini menunjukkan bahwa TextField pada Compose bersifat

state-driven, yaitu hanya menampilkan apa yang tersimpan di dalam state. Apabila text = it tidak dilakukan, state tidak berubah dan huruf yang diketik tidak akan muncul di layar.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat.

Jawaban:

Terdapat dua state pada program terakhir. State pertama adalah count bertipe Int dengan nilai awal 0, yang berfungsi menyimpan jumlah klik pada tombol “Tambah”. State kedua adalah text bertipe String dengan nilai awal kosong, yang berfungsi menyimpan isi dari Text Field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat.

Jawaban:

Terdapat dua event pada program terakhir, yaitu onClick pada Button “Tambah” dan onChange pada OutlinedTextField.

20. Kapan event-event tersebut dijalankan?

Jawaban:

Event onClick dijalankan setiap kali pengguna menekan tombol “Tambah”. Pada saat itu program menambah nilai count (count++) dan mencatat log “Counter berubah”. Event onChange dijalankan setiap kali isi Text Field berubah, baik karena pengguna mengetik, menghapus, maupun menempelkan teks. Pada saat itu program memperbarui state text (text = it) dan mencatat log “Text berubah”.

Percobaan 3: Implicit Intent

Percobaan ini menggunakan composable ImplicitIntentTest yang memiliki empat tombol, yaitu Buka Browser, Buka Telepon, Kirim Email, dan Buka Maps. Penomoran soal pada modul bagian ini tidak berurutan dengan rapi, sehingga jawaban dikelompokkan menjadi empat soal berdasarkan isinya, yaitu pengujian tombol, perbedaan action, fungsi data, dan hubungan antara action dan data.

1. Jalankan program dan uji setiap tombol (Buka Browser, Buka Telepon, Kirim Email, Buka Maps). Amati apa yang terjadi pada setiap aksi, lalu ambil screenshot hasilnya.

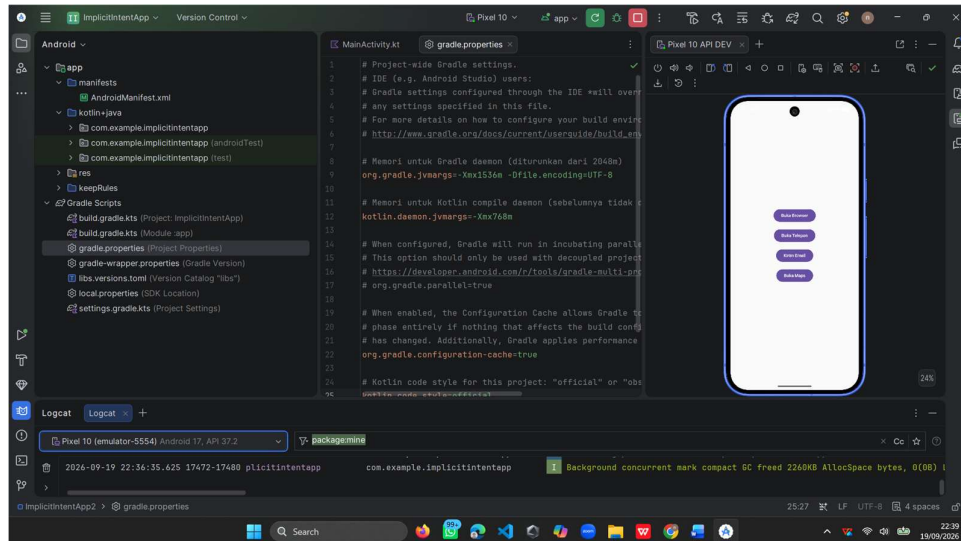
Jawaban:

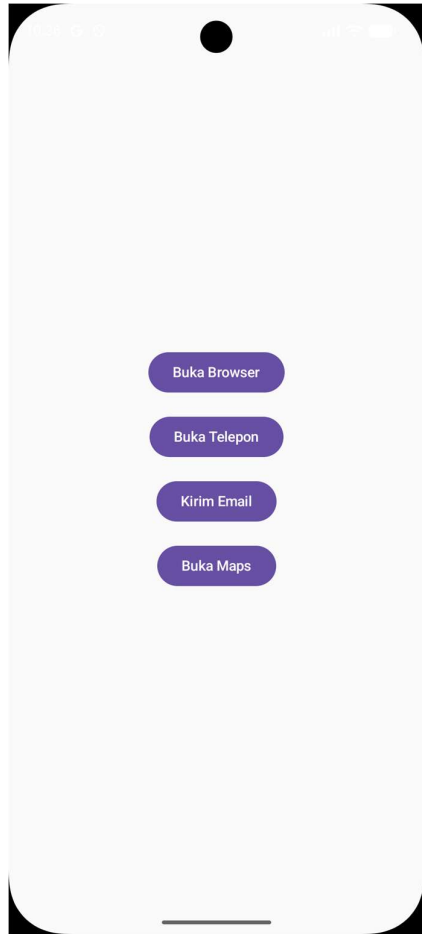
Pada tombol Buka Browser, aplikasi mengirim intent ACTION_VIEW dengan data <https://www.google.com>. Hasilnya, aplikasi browser terbuka dan menampilkan halaman Google.

Pada tombol Buka Telepon, aplikasi mengirim intent ACTION_DIAL dengan data tel:08123456789. Hasilnya, aplikasi telepon terbuka dengan nomor 08123456789 sudah terisi pada dialer. Panggilan belum dilakukan sampai pengguna sendiri menekan tombol panggil.

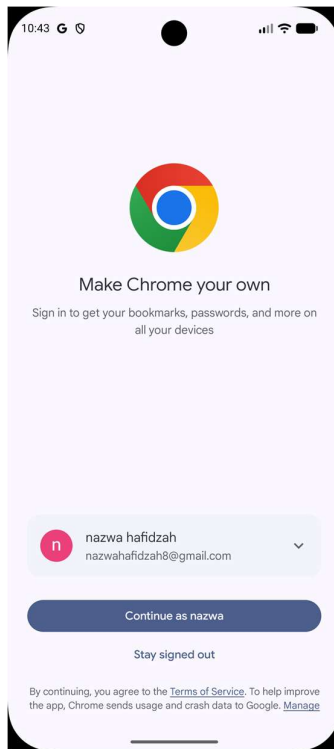
Pada tombol Kirim Email, aplikasi mengirim intent ACTION_SENDTO dengan data <mailto:test@gmail.com> yang dibungkus dengan Intent.createChooser. Hasilnya, muncul pilihan aplikasi email dengan judul “Pilih aplikasi email”. Setelah salah satu aplikasi dipilih, halaman penulisan email terbuka dengan penerima test@gmail.com.

Pada tombol Buka Maps, aplikasi mengirim intent ACTION_VIEW dengan data geo:-7.250445,112.768845. Hasilnya, aplikasi peta (misalnya Google Maps) terbuka dan menampilkan lokasi pada koordinat tersebut.

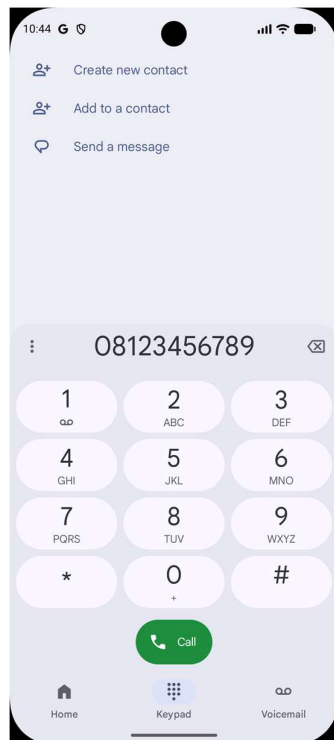




Gambar 1. Tampilan Aplikasi ImplicitIntentApp



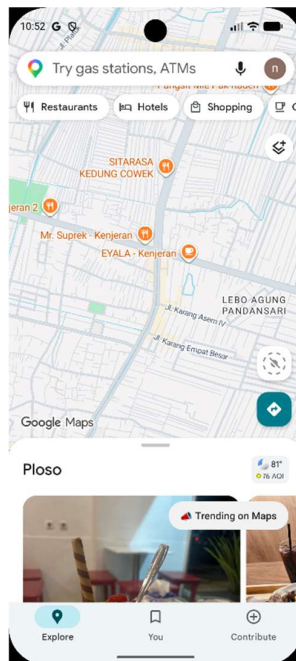
Gambar 2. tombol Buka Browser



Gambar 3. Hasil tombol Buka Telepon



Gambar 4. Hasil tombol Kirim Email



Gambar 5. Hasil tombol Buka Maps

Sebagai catatan, apabila perangkat atau emulator tidak memiliki aplikasi yang sesuai, misalnya belum ada aplikasi email atau peta, pemanggilan `startActivity()` akan melempar `ActivityNotFoundException` dan aplikasi dapat berhenti. Untuk mencegahnya, pemanggilan intent dapat dibungkus dengan try-catch.

2. Apa perbedaan fungsi dari masing-masing action intent: `Intent.ACTION_VIEW`, `Intent.ACTION_DIAL`, dan `Intent.ACTION_SENDTO`?

Jawaban:

`Intent.ACTION_VIEW` adalah action umum untuk menampilkan data kepada pengguna. Aplikasi yang membukanya bergantung pada jenis data yang dikirim bersamanya. Sebagai contoh, URL https dibuka di browser, sedangkan URI geo dibuka di aplikasi peta.

`Intent.ACTION_DIAL` berfungsi membuka aplikasi telepon dengan nomor yang sudah terisi tanpa langsung melakukan panggilan. Keputusan untuk menelepon tetap berada pada pengguna, sehingga action ini tidak memerlukan izin `CALL_PHONE`. Hal ini berbeda dengan `ACTION_CALL` yang langsung menelepon dan memerlukan izin tersebut.

`Intent.ACTION_SENDTO` berfungsi mengirim sesuatu kepada tujuan tertentu yang ditentukan oleh data URI. Pada kode praktikum, tujuannya adalah alamat email dengan skema `mailto:`, sehingga hanya aplikasi email yang dapat menanganinya.

3. Apa fungsi data pada intent? (contoh: `mailto:test@gmail.com`, `https://www.google.com`, `geo:-7.250445,112.768845`)

Jawaban:

Data pada intent, yang berbentuk URI, menyatakan objek atau sumber daya yang akan diproses oleh action. Data juga memberi tahu sistem jenis data yang dimaksud melalui skema URI, sedangkan isinya digunakan oleh aplikasi tujuan untuk menjalankan aksi tersebut.

Pada `mailto:test@gmail.com`, skemanya adalah `mailto` dan isinya adalah alamat email tujuan yang akan diisi sebagai penerima. Pada `https://www.google.com`, skemanya adalah `https` dan isinya adalah alamat halaman web yang akan dibuka. Pada `geo:-7.250445,112.768845`, skemanya adalah `geo` dan isinya adalah koordinat lintang dan bujur yang akan ditampilkan pada peta.

4. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Jawaban:

Implicit intent tidak menyebutkan aplikasi tujuan secara langsung. Sistem Android mencocokkan action dan data (skema URI) pada intent dengan intent-filter yang dideklarasikan oleh setiap aplikasi pada manifest-nya. Hanya aplikasi yang mendukung kombinasi action dan data tersebut yang menjadi kandidat untuk dibuka.

Sebagai contoh, kombinasi ACTION_VIEW dengan skema https ditangani oleh aplikasi browser, sedangkan ACTION_VIEW dengan skema geo ditangani oleh aplikasi peta. Kombinasi ACTION_DIAL dengan skema tel ditangani oleh aplikasi telepon, dan ACTION_SENDTO dengan skema mailto ditangani oleh aplikasi email. Hal ini menunjukkan bahwa action yang sama dengan data yang berbeda dapat menghasilkan aplikasi yang berbeda.

Apabila terdapat lebih dari satu aplikasi yang cocok, sistem menampilkan pilihan (chooser) agar pengguna dapat memilih, seperti yang terjadi pada tombol Kirim Email. Apabila tidak ada aplikasi yang cocok, terjadi ActivityNotFoundException.

Kesimpulan

Berdasarkan ketiga percobaan yang telah dilakukan, dapat disimpulkan beberapa hal berikut. Pendekatan imperatif pada XML/View mengubah tampilan dengan memerintahkan objek View secara langsung, sedangkan pendekatan deklaratif pada Compose mengubah state dan membiarkan Compose memperbarui tampilan melalui recomposition.

Variabel biasa tidak memicu perubahan tampilan. Agar tampilan berubah, data harus berupa state (mutableStateOf) dan dipertahankan menggunakan remember, sedangkan property delegation by membuat penggunaan state lebih ringkas karena tidak memerlukan .value. Event seperti onClick dan onChange menangani interaksi pengguna, dan pada Compose tugas utamanya adalah mengubah state sehingga tampilan mengikuti state tersebut.

Terakhir, implicit intent memungkinkan aplikasi membuka aplikasi lain, seperti browser, telepon, email, dan peta, hanya dengan menentukan action dan data. Pemilihan aplikasi yang sesuai dilakukan oleh sistem Android.